



Designing Self-Healing Airflow Platforms



Kumuda Sreenivasa · Sandeep Bommiseti

It is 03:14, and a human is doing what a machine could do

68%

of production failures were resolved by an action a machine could have taken - clear, wait, re-run one partition

34 min

median time from failure to first human action on the overnight rotation

6 min

median time the actual fix took, once somebody finally looked at it

“We did not have a failure problem. We had a response problem.”

03:14



task fails on a schema change

03:48



on-call acknowledges the page

03:54



clear + retry; it just works

Self-healing is not 'more retries'

NOT SELF-HEALING

`retries = 5`

Blind retries mask root cause and multiply load on the system that is already failing.

An alert with a runbook link

That is a notification with extra steps. A human is still the executor.

Auto-approving everything

Financial, regulatory and PII pipelines must never be repaired without a named human.

A model improvising a fix in prod

Non-deterministic remediation is an incident generator. Actions must be typed and enumerable.

SELF-HEALING

A closed control loop

Detect → classify → decide → act → verify → learn, with every step observable.

A bounded action vocabulary

~12 typed actions. Nothing outside the list can ever execute, whatever the input.

Policy as reviewed code

Remediation lives in git, gets a PR, gets unit tests, gets a blast-radius budget.

A kill switch and an audit trail

One flag disables autonomy platform-wide; every action is attributable and reversible.

Where is your platform today?

A

L0 • Manual

we retry by hand

B

L1 • Reactive

alerts and runbooks

C

L2 • Guarded

contracts, breakers

D

L3 • Autonomous

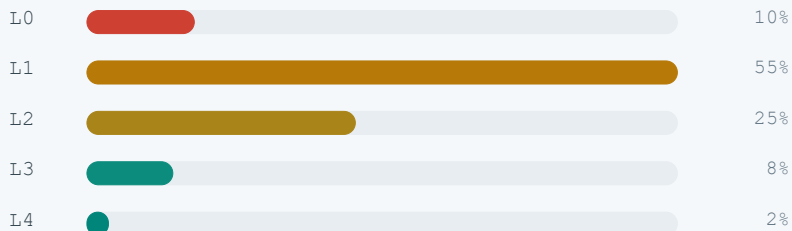
policy-driven repair

E

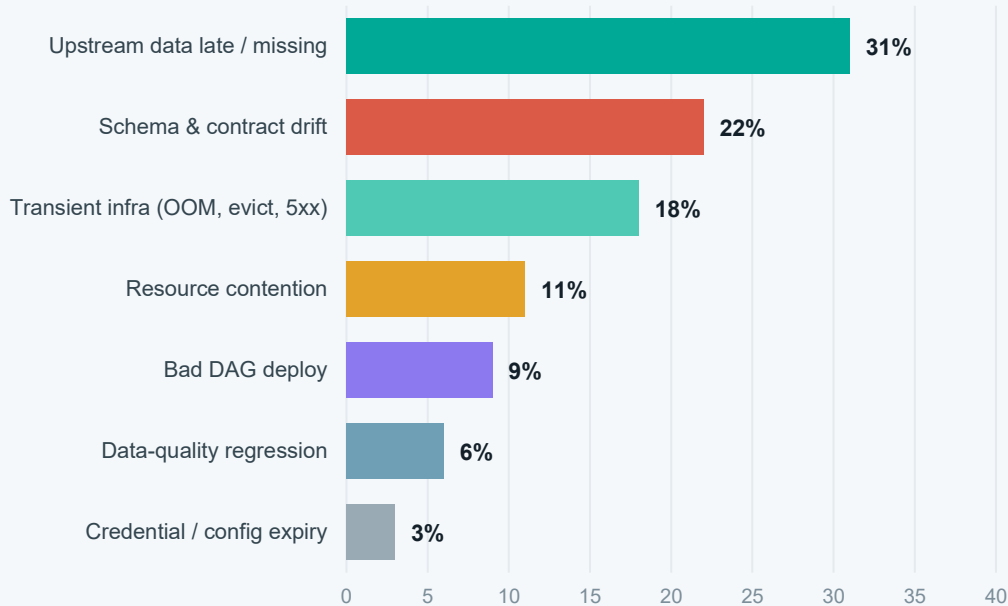
L4 • Predictive

risk-scored, canaried

What a room like this usually says



What actually breaks: 12 months across both platforms



63% is mechanically recoverable

Late data, transient infra and contention all have deterministic, safe remediations. This is the automation budget.

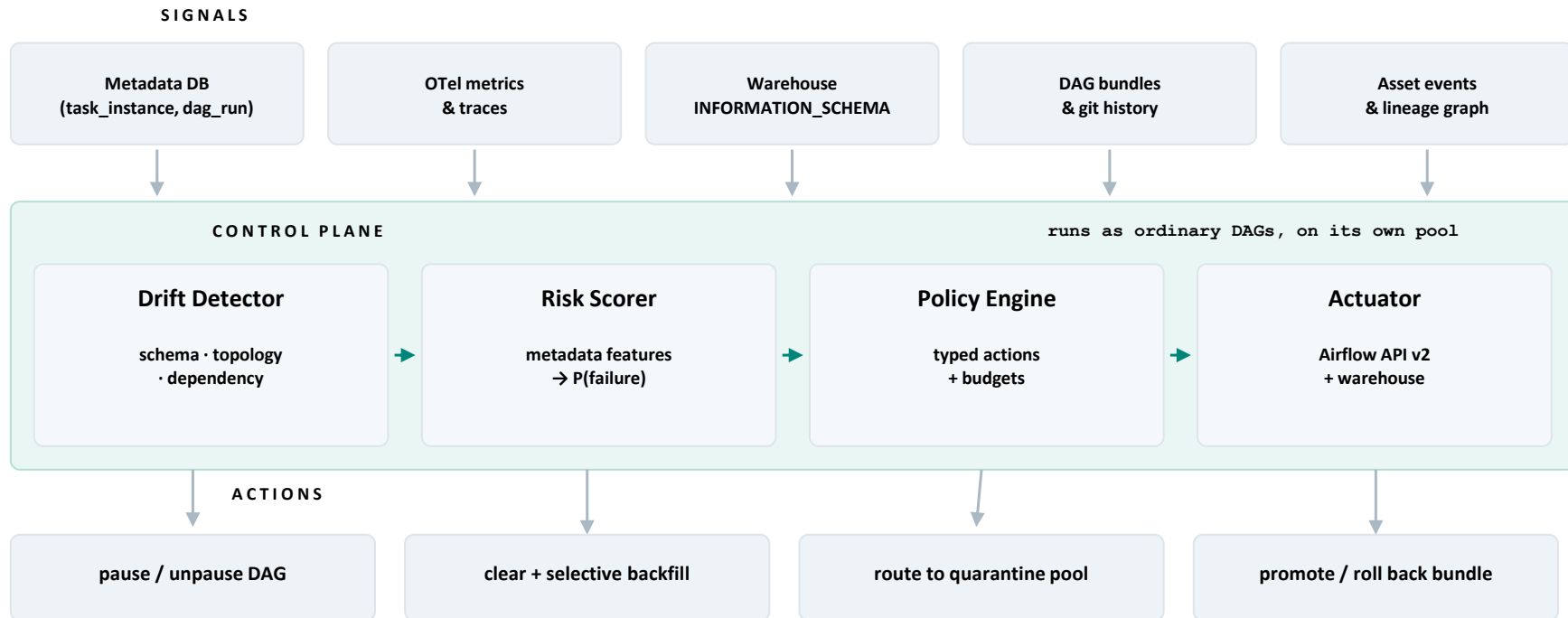
Schema drift is the silent killer

22% of failures, but 61% of the total minutes lost - it fails late, downstream, and often silently succeeds with wrong data.

9% is a deploy problem

Bad DAG changes should never have reached every run at once. That is a release-engineering fix, not a recovery fix.

Airflow as an intelligent control plane



Every component is a DAG. No sidecar service to operate, no second scheduler to keep alive - the control plane is scheduled by the thing it controls.

Where the loop hooks into Airflow

AIRFLOW COMPONENT	THE SURFACE WE USE	WHAT THE CONTROL PLANE ATTACHES
DAG processor & bundles	post-parse hook, bundle git ref	structural fingerprints, canary staging
Scheduler, timetables, assets	asset events, deadline alerts (3.1)	late-upstream detection, freshness features
Executor, pools & queues	pool slots, priority_weight, per-team executors (3.2)	tiering and the quarantine lane
Task SDK listeners	on_task_instance_failed / _success	classify, trip breakers, start the loop
Metadata DB (read replica)	task_instance, dag_run, job	risk features, retraining set, audit joins
API server, v2	/dags, /dagRuns, /backfills	the actuator's only door - nothing writes directly
Task store (3.3)	durable per-task state	breaker and canary state, without abusing Variables

One listener starts the whole loop

```
# plugins/healing_listener.py - a plugin, not a sidecar service
from airflow.listeners import hookimpl
from airflow.plugins_manager import AirflowPlugin

class HealingListener:
    @hookimpl
    def on_task_instance_failed(self, previous_state, task_instance,
                               error, session=None):

        try:
            breaker.record(task_instance.dag_id, ok=False)
            cls = classify(task_instance, error) # 9 error classes
            decision = policy.decide(task_instance, cls)
            audit.write(decision) # always, even "none"
            if decision.action != "none":
                actuator.enqueue(decision) # never act inline
        except Exception:
            log.exception("healing listener failed open")

    @hookimpl
    def on_task_instance_success(self, previous_state,
                                task_instance, session=None):
        breaker.record(task_instance.dag_id, ok=True)

class HealingPlugin(AirflowPlugin):
    name = "self_healing"
    listeners = [HealingListener()]
```

Listeners are the supported seam

No monkey-patching, no polling loop. `on_dag_run_failed` and asset events give you the other two entry points.

Enqueue, never act inline

This runs inside the worker or scheduler process. Anything slow or networked here becomes your next incident.

Fail open, always

A bare except around the whole body is correct here. The healer must never be able to fail a task that would have passed.

Audit even when you do nothing

Writing the 'no action' decision is what lets you prove later that the loop considered an incident and declined.

Schema drift: fail at the boundary, not in the warehouse

```
# plugins/healing/contracts.py
from airflow.sdk import task
from airflow.exceptions import AirflowFailException

# severity ladder - additive change is not an outage
RANK = {"ADDITIVE": 0, "WIDENED": 1, "RENAMED": 2,
        "TYPE_CHANGED": 3, "DROPPED": 3}

@task(retries=0, pool="control_plane", multiple_outputs=True)
def check_contract(table: str, **ctx) -> dict:
    expected = registry.load(table) # versioned in git
    observed = warehouse.describe(table) # INFORMATION_SCHEMA
    diff = expected.diff(observed)
    if not diff:
        return {"status": "clean", "diff": []}

    worst = max(RANK[d.kind] for d in diff)
    emit_drift_event(table, diff, ctx["dag_run"].run_id)

    if worst >= 3:
        quarantine(table, diff) # isolate, do not cascade
        raise AirflowFailException(f"drift: {diff.summary()}")
    if worst == 2:
        apply_column_map(table, diff) # rename: mappable
    return {"status": "degraded", "diff": diff.as_dict() }
```

Contract lives in git

Reviewed like code, versioned with the DAG bundle, and diffed in CI before it ever runs.

Severity decides autonomy

Additive and widened changes heal themselves. Drops and type changes never do.

Quarantine ≠ fail

The table is marked stale so 11 downstream DAGs skip instead of writing wrong numbers.

Run it as a sensor-cheap gate

One query per table per run, ~40 ms. Cheaper than one failed Spark job by five orders of magnitude.

Structural fingerprints: catch drift the parser is happy with

```
# plugins/healing/fingerprint.py
import hashlib, json

def dag_fingerprint(dag) -> str:
    """Stable hash of everything that changes behaviour."""
    spec = {
        "tasks": sorted(
            (t.task_id, type(t).__name__, t.pool, t.queue)
            for t in dag.tasks),
        "edges": sorted(
            (t.task_id, d)
            for t in dag.tasks for d in t.downstream_task_ids),
        "assets": sorted(a.uri for a in inlet_assets(dag)),
        "schedule": str(dag.timetable.serialize()),
        "libs": pinned(("pandas", "pyarrow", "dbt-core")),
    }
    blob = json.dumps(spec, sort_keys=True).encode()
    return hashlib.sha256(blob).hexdigest()[ :16]

def on_bundle_parsed(dag) -> None:          # post-parse hook
    prev, now = registry.promoted(dag.dag_id), dag_fingerprint(dag)
    if prev and prev != now:
        registry.stage(dag.dag_id, now, mode="canary") # not live yet
```

The parse succeeded. That proves nothing.

A task moved between pools, an edge disappeared, pyarrow went 17→18. All valid Python. All incidents.

Fingerprint = the deploy gate

A changed hash routes the version to canary instead of straight to every scheduled run.

Pin what you hash

Library versions belong in the fingerprint. Half of 'it worked yesterday' is a transitive dependency.

Pairs with DAG versioning

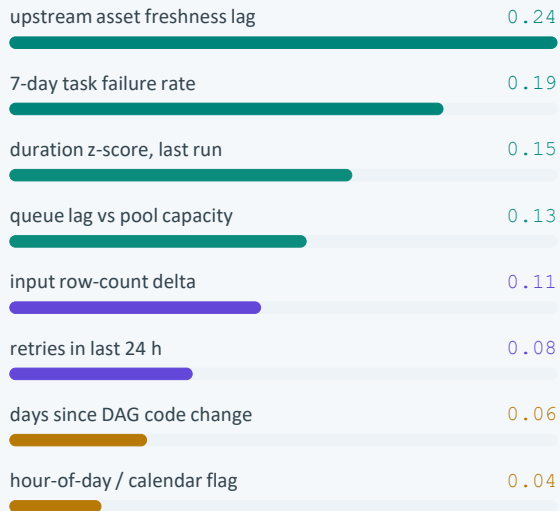
Airflow 3 records which version produced each run, so you can diff behaviour across versions, not just code.

Your metadata DB already knows which run will fail

```
-- features per (dag_id, task_id, hour) from the metadata DB
-- run against a READ REPLICAs. never the scheduler's primary.
WITH ti AS (
  SELECT dag_id, task_id, run_id, state, try_number,
         EXTRACT(EPOCH FROM (end_date - start_date)) AS dur_s,
         EXTRACT(EPOCH FROM (start_date - queued_dttm)) AS q_lag_s,
         EXTRACT(HOUR FROM start_date) AS hod
  FROM task_instance
  WHERE start_date > now() - interval '90 days'
  AND state IN ('success', 'failed', 'up_for_retry')
)
SELECT dag_id, task_id, hod,
       AVG(dur_s) AS dur_mean,
       STDDEV_POP(dur_s) AS dur_std,
       AVG(q_lag_s) AS q_lag_mean,
       AVG(CASE WHEN state='failed' THEN 1 ELSE 0 END) AS fail_rate,
       AVG(try_number - 1) AS retry_rate,
       COUNT(*) AS n_runs
FROM ti GROUP BY 1, 2, 3
HAVING COUNT(*) >= 20;
```

Gradient-boosted model, 8 features

relative importance, retrained weekly



No new infrastructure: every feature above is a query away in the database Airflow is already writing to. Start there before buying anything.

Isolate unstable workloads before they spread

Tier 0 · Critical

dedicated pool + own executor (multi-team, 3.2)
never preempted, never shares a node pool

worker slots

16 slots



Tier 1 · Standard

shared pool, per-DAG max_active_tasks cap
priority_weight fixed by tier, not by author

worker slots

22 slots



Tier 2 · Batch & backfill

low priority, preemptible nodes, off-peak window
self-healing replays land here by default

worker slots

12 slots



Quarantine

4 slots, no asset emission, no shared connections
a DAG in here cannot trigger anything downstream

worker slots

4 slots



Tiering is a contract, not a config: a DAG earns Tier 0 by having an owner, an SLA and a rollback plan. Everything else starts at Tier 1 and can be demoted automatically.

Circuit breakers: stop paying for a failure you already understand

```
# plugins/healing/breaker.py - state in the task store (3.3)
from datetime import timedelta
WINDOW, THRESHOLD, COOLDOWN = 10, 0.5, timedelta(minutes=30)

def record(dag_id: str, ok: bool) -> None:
    key = f"breaker:{dag_id}"
    s = store.get(key, default={"runs": [], "opened_at": None})
    s["runs"] = (s["runs"] + [1 if ok else 0])[-WINDOW:]

    rate = 1 - sum(s["runs"]) / len(s["runs"])
    tripped = len(s["runs"]) == WINDOW and rate >= THRESHOLD
    if tripped and s["opened_at"] is None:
        pause_dag(dag_id, why=f"breaker open: {rate:.0%} failing")
        s["opened_at"] = utcnow()
    store.set(key, s)

def half_open_sweep() -> None: # control DAG, every 5 minutes
    for dag_id, s in store.scan("breaker:"):
        if s["opened_at"] and utcnow() - s["opened_at"] > COOLDOWN:
            unpause_dag(dag_id) # exactly one probe run decides
            s["runs"], s["opened_at"] = [], None
```

Retries amplify. Breakers absorb.

A DAG failing on a dead credential retried 4,100 times in one night. The breaker version stops after 10.

Half-open, not fully open

One probe run after cooldown. Recovery is proven by a real run, never assumed by a timer.

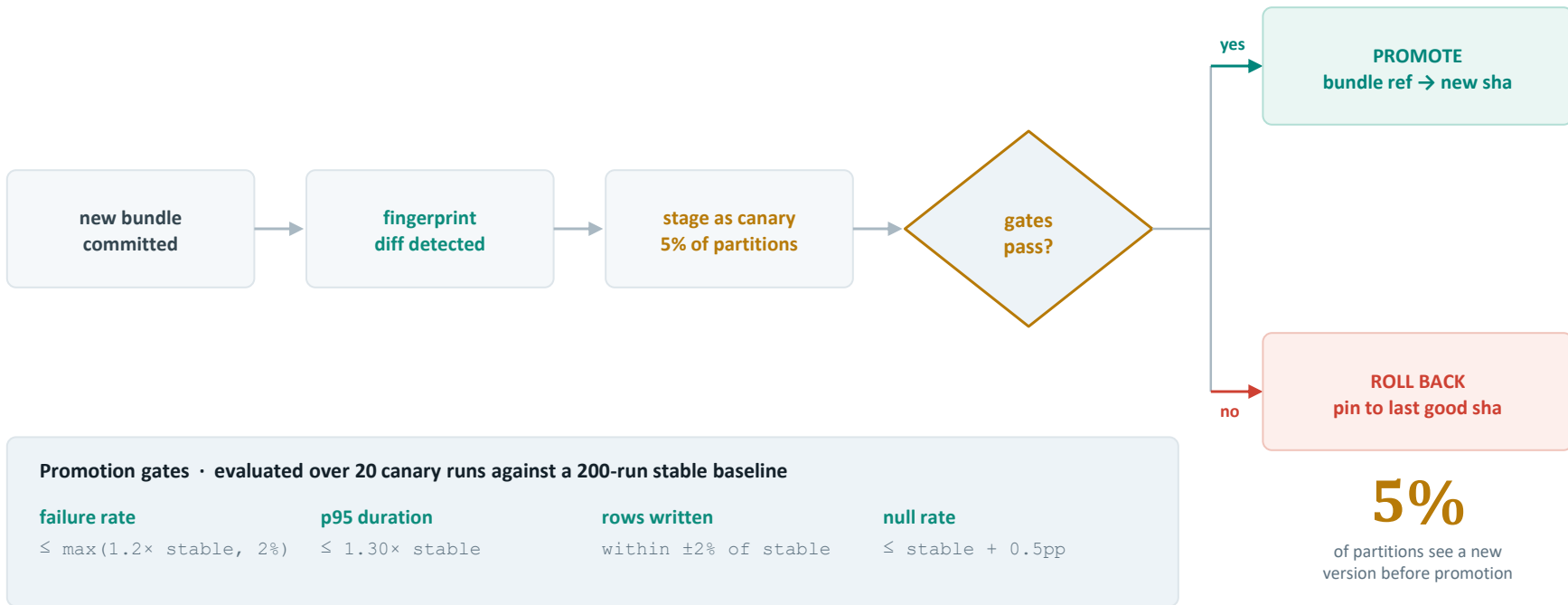
Trip on rate, not on count

Count-based breakers punish high-frequency DAGs and ignore the daily one that has failed three days running.

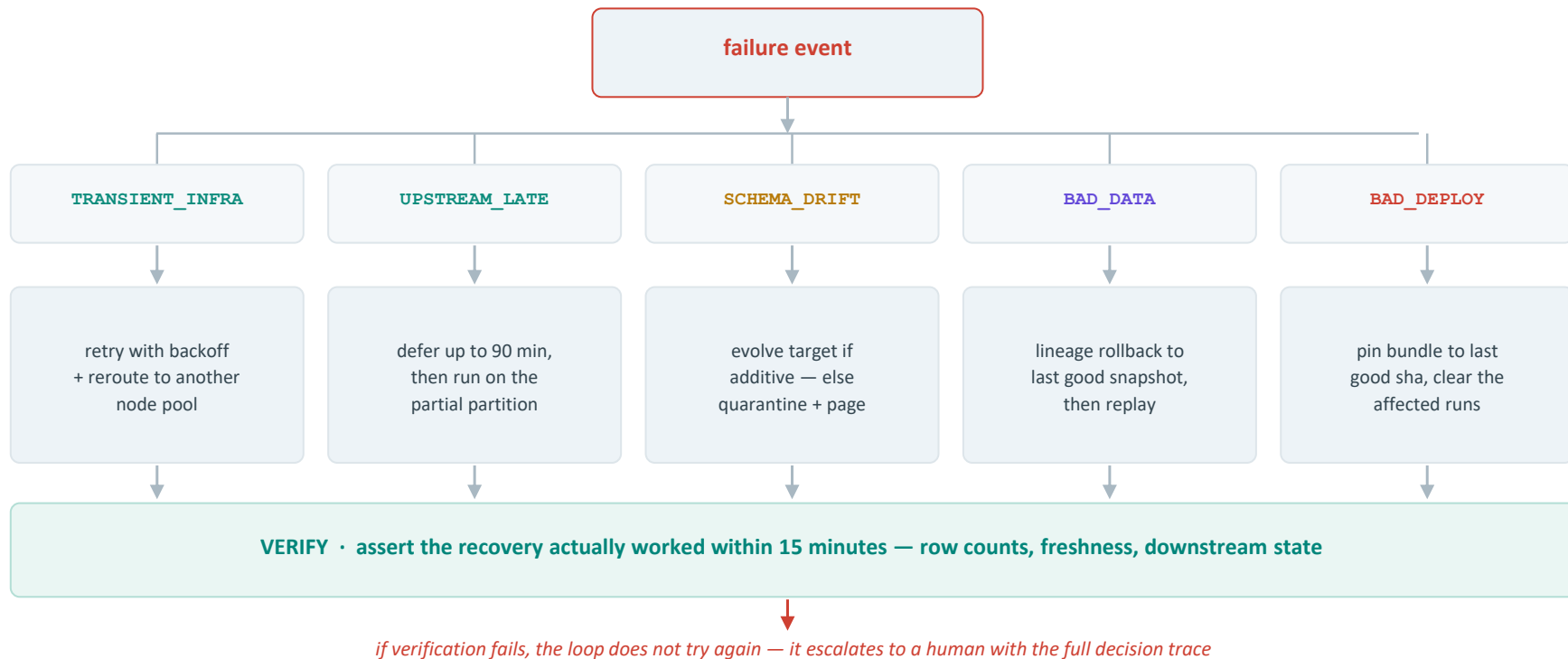
Always page when you trip

Opening a breaker is a decision to stop delivering data. That is an incident, even if nothing is on fire.

Canary deployment for DAGs



One failure event, nine classes, twelve legal actions



Pick a failure and we will walk it end to end

TRANSIENT INFRA

"the pod died again"

Isolation & pools

[slide 14 →](#)

UPSTREAM LATE

"the vendor is four hours late"

Policy as code

[slide 19 →](#)

SCHEMA DRIFT

"a column changed type"

Contracts

[slide 11 →](#)

BAD DATA

"the numbers are wrong"

Lineage rollback

[slide 21 →](#)

BAD DEPLOY

"yesterday's merge did it"

Canary DAGs

[slide 16 →](#)

Whatever you pick, the machinery is identical

classify the error → look up the rule → check the blast-radius budget → take exactly one typed action → verify inside 15 minutes → escalate if verification fails.
That sameness is the point: nine error classes, twelve actions, one code path to test. A tenth class means writing one rule, not building another system.

Remediation policy belongs in git, not in a runbook

Reviewed like production code

A pull request, two approvals, and a unit test per rule that replays a recorded incident against it.

Budgets are the safety net

Caps on actions per hour, rows reprocessed and cascade depth. Exceed a budget and the loop stops and pages.

never_touch is absolute

Financial, regulatory and PII pipelines are excluded by glob. No policy, model or override can reach them.

Versioned with the decision

Every audit row records the policy sha that made the call, so you can explain a bad decision six months later.

```
# policies/orders_pipeline.yml
dag_id: orders_pipeline
budget: # blast-radius guardrails
  max_auto_actions_per_hour: 3
  max_rows_reprocessed: 50_000_000
  max_cascade_depth: 4
  never_touch: ["finance_gl_*", "**_regulatory_*"]

rules:
- when: {class: TRANSIENT_INFRA, retries_left: ">0"}
  do: retry_with_backoff
  args: {max_delay: 15m, reroute_pool: spillover}

- when: {class: UPSTREAM_LATE, asset: "orders.raw"}
  do: defer_run
  args: {max_wait: 90m, then: run_partial_partition}

- when: {class: SCHEMA_DRIFT, severity: additive}
  do: evolve_target_schema
  args: {mode: add_columns_only, then: replay_partition}

- when: {class: SCHEMA_DRIFT, severity: breaking}
  do: quarantine_and_page # never auto-healed

- when: {class: BAD_DATA, check: row_count_drop}
  do: lineage_rollback
  args: {to: last_good_snapshot, approval: required}
```

Selective backfill: replay what is dirty, not what is convenient

August partitions, orders_fact

after a 41-day contract violation on the ERP feed



dirty — replayed

verified clean — untouched

16 partitions → 3 contiguous windows → 11% of a full-month replay

```
# control_plane/replay.py — Airflow 3 backfill API
def replay(dag_id: str, dirty: list[date]) -> list[str]:
    assert sink_is_idempotent(dag_id) # MERGE, not INSERT
    windows = merge_contiguous(dirty) # 16 days -> 3 windows
    return [client.post("/api/v2/backfills", json={
        "dag_id": dag_id,
        "from_date": start.isoformat(),
        "to_date": end.isoformat(),
        "reprocess_behavior": "failed", # leave good runs
        "max_active_runs": 4, # bounded concurrency
        "dag_run_conf": {"trigger": "self_heal",
                        "incident": incident_id,
                        "backfill_id": backfill_id}
    }).json()["backfill_id"] for start, end in windows]
```



idempotent sink

MERGE on a natural key, or the replay doubles your revenue



bounded concurrency

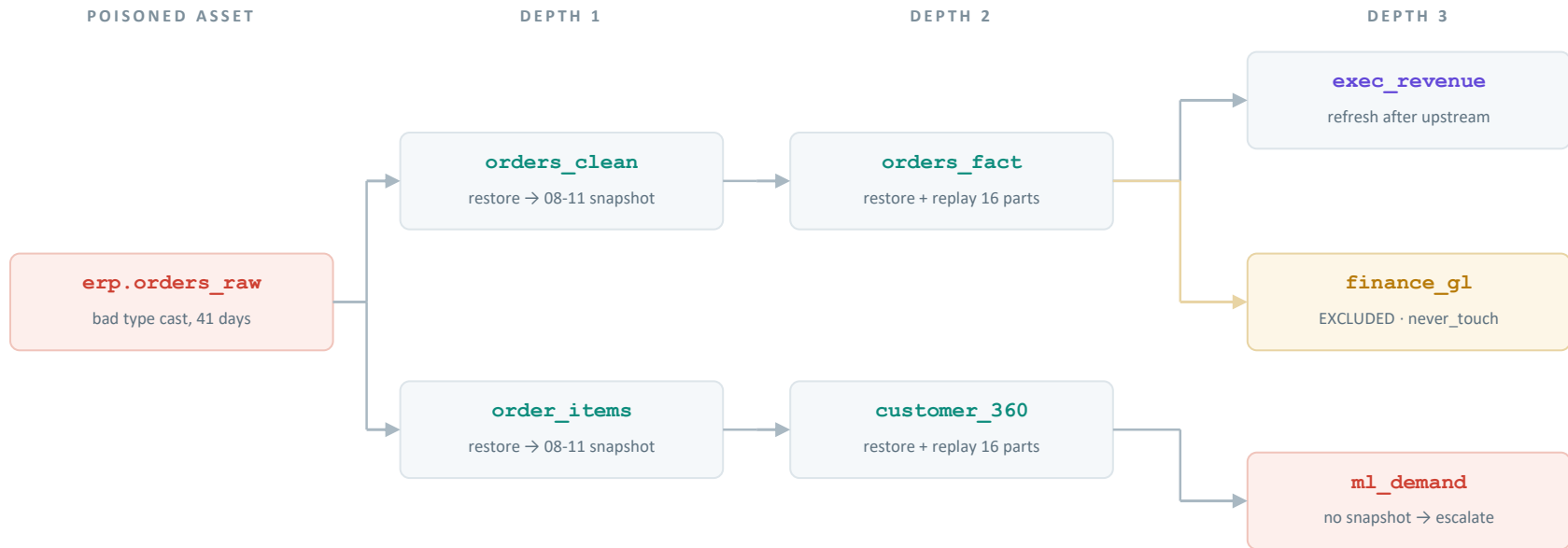
max_active_runs stops the repair from becoming the outage



reprocess_behavior

'failed' leaves good runs alone — the default is not always what you want

Lineage-aware rollback: the blast radius is a graph



Topological order, always

Restore parents before children, or you replay a child against data that is still wrong.

Excluded nodes stop the cascade

finance_gl is on never_touch, so the plan halts there and raises a ticket with the exact diff.

No snapshot → no autonomy

An asset without a recoverable point-in-time is escalated, never guessed at. Snapshots are a prerequisite.

Where autonomy has to stop

NEVER AUTO-HEAL

✗ Anything a regulator reads

Financial ledgers, statutory reports, audit tables. Excluded by glob, enforced in the policy engine, tested in CI.

✗ Non-idempotent writes

If the sink cannot absorb the same rows twice, a replay is a data-loss event with extra steps.

✗ Unknown error classes

If the classifier is not confident, the action is 'page a human with the trace'. Unclassified never means retry.

✗ A failed remediation

One attempt. Verification decides. There is no second-order autonomy, ever.

✗ Anything during a declared incident

When a human takes command, the loop goes read-only and keeps producing evidence, not actions.

Human-in-the-loop is a first-class Airflow task (3.1+)

```
from airflow.providers.standard.operators.hitl \
    import ApprovalOperator

approve = ApprovalOperator(
    task_id="approve_rollback",
    subject="Roll back finance_gl to the "
           "2026-08-11 snapshot?",
    body=plan.render_diff(),      # tables, rows,
                                # owners, cost
    defaults="Reject",           # timeout = no-op
    execution_timeout=timedelta(minutes=20),
)
```

The default must be 'Reject'

A timed-out approval has to fail closed. If nobody answers at 3 a.m., the correct outcome is a stale table and a page — not an unreviewed rollback of the general ledger.

The control plane is a privileged actor

WHAT IT IS TRUSTED TO DO

pause / unpause any DAG

clear + backfill any run

restore tables from snapshots

re-point bundle refs

THREAT	VECTOR	BLAST RADIUS	CONTROL
Stolen actuator token	one global DAG-write credential	pause everything, mass-clear runs	one scoped identity per action class, minutes-long TTL
Policy tampering	a pull request to policies/*.yaml	never_touch removed, budgets widened	CODEOWNERS + signed commits; policy sha in every audit row
Poisoned DAG bundle	the canary promotion path	code runs with the worker identity	protected branch, signed tags, second human on Tier 0
Feature / metric poisoning	writes to the metadata or metrics store	forced quarantine, or detection silenced	read-only replica, drift caps, shadow-mode fallback
Automation as amplifier	repeatedly tripping remediation	self-inflicted DoS and cost blowout	budgets are rate limits; put a breaker on the healer too

Anything that can repair production can damage it. Model the healer as an insider with root, because that is exactly what it is.

Airflow's own attack surface, now with automation on top

AIRFLOW SURFACE

WHAT GOES WRONG

WHAT WE DO ABOUT IT

dag_run.conf

The healer passes conf into automated backfills, and conf reaches Jinja, operator args and SQL.

Validate against a schema and allow-list the keys. Never template conf into a shell or a query.

Connections & Variables

One compromised DAG can read every connection the worker can reach — a credential sweep, not an outage.

Secrets backend, per-team connections (3.2), no credentials in Variables, rotate the Fernet key.

DAG code itself

A bundle write is arbitrary code execution as the worker identity. Canary promotion is a path into prod.

Protected branch, CODEOWNERS, signed commits, and a second human before Tier 0 promotion.

Task logs

Remediation logs happily print the rows they repaired — including the PII you are not allowed to keep.

Redact at the logger, remote logging to a restricted bucket, short retention on repair logs.

API tokens

The actuator holds what is effectively a DAG-write credential for the whole deployment.

One scoped token per action class, minutes-long TTL, issued separately from human SSO.

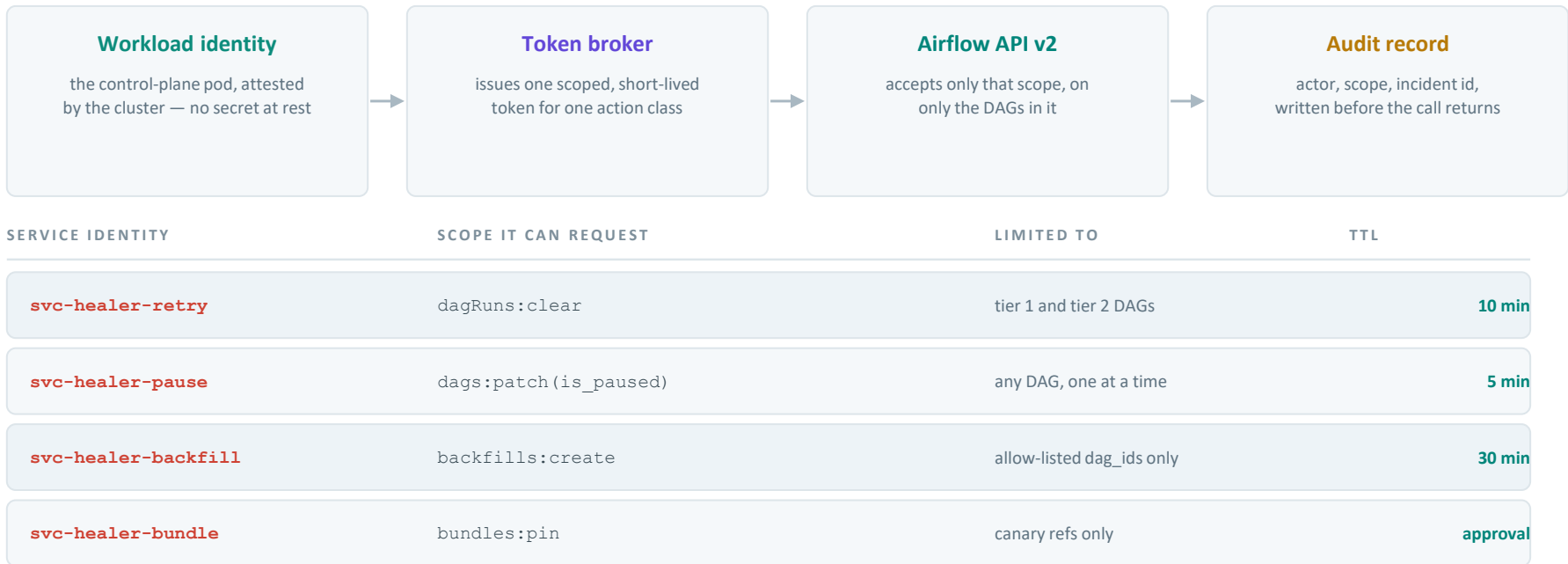
Plugins & listeners

Listener code is loaded into the scheduler process. A malicious plugin owns the control plane.

Review plugins like production code; never import listener logic from a Variable or a remote path.

None of these are new in Airflow 3 — they are the surfaces you already have. Automation just raises the value of exploiting them.

The actuator gets an identity per action, not a password



No identity holds both backfill and bundle. Separation of duties is why a single stolen token cannot rewrite history and then hide the code that did it.

Nothing parses unless it is signed

```
# plugins/healing/bundle_guard.py - runs before the DAG is parsed
ALLOWED_SIGNERS = {"platform-ci@atc", "release-bot@receptly"}

def verify_bundle(ref: str) -> None:
    meta = registry.fetch(ref)

    if not cosign.verify(meta.digest, meta.signature, PUBLIC_KEY):
        raise BundleRejected(f"unsigned or tampered: {ref}")

    if meta.signer not in ALLOWED_SIGNERS:
        raise BundleRejected(f"unknown signer: {meta.signer}")

    if meta.tier == 0 and meta.pr_approvals < 2:
        raise BundleRejected("tier 0 requires two approvals")

    pins.assert_locked(meta.requirements)  # no floating versions
    sbom.assert_no_known_critical(meta.sbom)

# the healer may pin an already-verified ref. it may never sign one.
def promote(ref: str) -> None:
    verify_bundle(ref)
    api.patch_bundle_ref(ref, token=scoped("bundles:pin"))
```

Canary promotion is a write path into production

The nicest thing about automated promotion is also the problem: it moves code without a human. Signature verification is what makes that survivable.

Verify before parse, not after

Parsing a DAG executes its module-level code. By the time you can inspect the object, the attacker has already run.

The healer pins, it never signs

Signing lives in CI with a key the runtime cannot reach. The control plane can only choose between refs that are already trusted.

Pins and SBOM in the same gate

Half of 'it worked yesterday' is a transitive dependency and so is a meaningful share of supply-chain compromise.

The Airflow settings that actually carry the risk

Secrets

- ✓ A real secrets backend — Vault, Secrets Manager, or the cloud equivalent
- ✓ Nothing sensitive in Variables; they are readable by any task that runs
- ✓ Fernet key rotation on a schedule you have actually rehearsed
- ✓ Per-team connection scoping so a Tier 2 DAG cannot reach a Tier 0 credential

Access

- ✓ DAG-level access control, not one blanket role for everyone
- ✓ The healer gets its own role, not Admin — and never a human's session
- ✓ Config endpoint not exposed; the UI leaks more than people expect
- ✓ Worker service accounts scoped per node pool, quarantine tightest of all

Evidence

- ✓ Remote logging to storage the workers cannot delete from
- ✓ Audit records shipped off-cluster, append-only, 400-day retention
- ✓ OTel traces on every automated action, correlated by incident id
- ✓ Alert when the healer's own token is used from an unexpected identity

And treat `dag_run.conf` as untrusted input — because with a healer in the loop, it is

```
CONF_SCHEMA = {"trigger": {"enum": ["self_heal", "manual", "canary"]},
               "incident": {"pattern": r"^INC-\d{6}$"}}

def safe_conf(conf: dict) -> dict:
    jsonschema.validate(conf, CONF_SCHEMA)           # reject, don't sanitise
    return {k: conf[k] for k in CONF_SCHEMA}         # allow-list, not filter
```

The control plane is already a security sensor

Same classifier, same policy engine, same audit trail — six extra signals and a different set of actions.

SIGNAL	WHAT IT CATCHES	AUTOMATED RESPONSE
Action from an unexpected identity	a stolen or misused healer token	freeze the loop, rotate, page security
Remediation burst outside business hours	the healer used as a living-off-the-land tool	budget trips; further actions need approval
conf that fails schema validation	injection through dag_run.conf	reject, quarantine the trigger source, alert
Bundle ref moved with no signed tag	an unauthorised deploy	refuse to parse, pin to last good, page
Connection reads spiking from one DAG	credential sweep from a compromised task	quarantine the DAG, revoke its session
Lineage queried by a role that owns nothing	reconnaissance before an exfiltration	silent deny under RLS, audit, no error text

You already built the detect-classify-decide-act loop. Pointing it at security signals costs six rules, not a second platform.

Five flaws in a policy that would pass review

```
dag_id: orders_pipeline
budget:
  max_auto_actions_per_hour: 50
  max_rows_reprocessed: 50_000_000
  never_touch: ["finance_gl"]

rules:
- when: {class: UNKNOWN}
  do:    retry_with_backoff

- when: {class: BAD_DATA}
  do:    lineage_rollback
  args:  {to: "{ dag_run.conf.snapshot }}" }

- when: {class: SCHEMA_DRIFT, severity: breaking}
  do:    evolve_target_schema
```

FIVE THINGS WRONG WITH IT

- 1 A number** the hourly action budget
- 2 A glob** that is not there
- 3 A default** for the unknown case
- 4 An input** that chooses the snapshot
- 5 A severity** that should never auto-heal

This file had two approvals on it. Reviewing a policy is not the same as reviewing code that runs once — every line here is a standing permission.

All five, and what each one costs you

1

`max_auto_actions_per_hour: 50`

A budget that high is not a guardrail, it is a rate-limit-shaped rubber stamp. Fifty actions an hour is how a retry storm looks from the inside.

→ 3 per hour, and breaker actions count against it

[s27](#)

2

`never_touch: ["finance_gl"]`

The glob is missing. `finance_gl_2026`, `finance_gl_staging` and `finance_gl_eu` are all fair game to the healer as written.

→ "finance_gl*" and a CI test that asserts the match set

[s19](#)

3

`UNKNOWN → retry_with_backoff`

An unclassified error is the one case where you know least. Retrying it is guessing with production. The default must fail closed.

→ UNKNOWN → quarantine_and_page, always

[s17](#)

4

`to: {{ dag_run.conf.snapshot }}`

Attacker-controllable input choosing which snapshot to restore. Anyone who can trigger a run picks the data your warehouse reverts to.

→ Allow-list the conf; resolve snapshots server-side

[s24](#)

5

`breaking_drift →
evolve_target_schema`

A dropped column or a changed type is not something to evolve into. This rule quietly rewrites the target to match whatever upstream did.

→ Breaking severity quarantines and pages. Never evolves.

[s11](#)

Every one of these shipped in a real policy file. Four were caught in review; the first one was not.

A silent OEM type change, six hours of wrong grades

The environment

12,000

worn cores received, scanned and graded per day

6

source systems, 40+ events per core

3

OEM feeds — and 3 to 6 schema changes each, per year, unannounced

< 4 h

QA-to-shipping SLA, warranty traceability

ITAR + GDPR

chain of custody; VIN is personal data in the EU dealer network

Infoflo/Unix ERP → SQL Server → Snowflake · 6 plants across OK, MI and TN

The Stellantis feed incident · 06:14, a Monday



06:14

Stellantis EDI ships condition_grade as TINYINT instead of VARCHAR(1). No notice, no version bump.



06:15

The implicit cast succeeds. A, B, C and D all land as 0. The pipeline is green. Zero alerts.



07:30

Grading logic reads 0 as unknown and routes every core to the manual review queue.



09:45

A floor supervisor notices the queue at 847 items. Normal is about 40. Not a monitor - a person.



10:22

Root cause found. 6 h 08 m of corrupt component_assessment rows, three pipelines contaminated.



12:40

Four tables rolled back by hand, two reruns, 847 cores re-graded. 6.4 hours end to end.

Nothing failed. That is exactly why it took three and a half hours to notice.

The one the canary caught: machine M-07

1 Batch lands, 06:00

847 QA records staged from seven load-test machines at shift start.

2 Canary samples 2%

A deterministic, machine-stratified slice runs the full pipeline in shadow.

3 torque_zscore fails

Three M-07 rows score 3.1 against a 30-day per-machine baseline. Threshold is 2.5.

4 Breaker opens, 06:04

The qa_results pipeline stops. On-call gets the machine id, the rows and the scores.

5 M-07 pulled, 06:18

Sensor confirmed drifting +8.3% over three days. Six machines keep running.

6 Selective replay, 14:30

After recalibration, only M-07 rows for that batch are replayed from raw telemetry.

22 of 22

acceptance checks passed — torque was in spec, and wrong

4 min

from batch landing to the machine being named

847

QA records that never reached shipping

6 of 7

machines that carried on working

The assertion was twenty minutes of SQL against a rolling per-machine baseline. That is the whole predictive model.

What changed at ATC Drivetrain

6.4 h → 23 min

recovery on a schema-drift incident

4 sec

to detect the type change, versus 3.5 hours

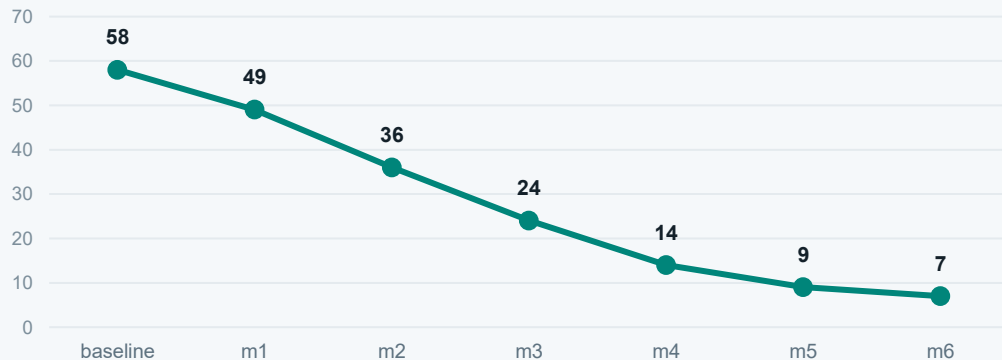
0

corrupt rows reaching downstream consumers

94%

average reduction in recovery time

Unplanned human interventions per week



Failure count did not drop. Detection improved, so we see more of them — they just stopped costing anything.

Weeks 1-6 · See everything

Contracts on the three OEM feeds, DAG fingerprints, features on a replica. MTTR halved before anything was automated.

Weeks 7-14 · Contain

Pool tiers, a quarantine lane, breakers on every DAG, canary promotion on bundle changes.

Weeks 15-24 · Close the loop

Policy engine with budgets, selective backfill, lineage rollback, HITL on anything irreversible.

The same patterns with two engineers and forty DAGs

The environment

9 POS partner integrations across Singapore

400+ merchant tenants sharing the same pipelines

RX-Bridge ESC/POS interceptors reading the receipt at the terminal

seconds receipt-to-app latency — the product is the freshness

PDPA purchase history is personal data, per tenant

40 DAGs · 2 engineers · one shared Airflow · no on-call rotation to speak of

A POS vendor firmware update, overnight

- **09:02** A partner pushes firmware to one merchant chain. The ESC/POS byte layout moves the tax line.
- **09:03** The parser still returns a receipt. Totals arrive null - for that chain only.
- **09:05** The per-tenant contract check trips on null_rate for tenant 118. Three minutes, not three hours.
- **09:05** Tenant 118 goes to the quarantine lane. The other 399 merchants keep publishing.
- **11:40** Vendor confirms the layout change. The parser fix ships behind a canary on 5% of that tenant.
- **12:10** Three hours of receipts replayed from the raw ESC/POS blob store, keyed by receipt hash.

Identical failure shape to Stellantis. The difference is that the blast radius here is a tenant, not a table.

What a two-person team should actually build

BUILD THESE THREE

- 1 Contracts per POS partner**
Nine partners, nine declared payload shapes. The whole detection story for a week of work.
- 2 Quarantine by tenant**
One merchant can fail without the other 399 noticing. Cheapest containment you will ever buy.
- 3 Replay from the raw store**
Keep the bytes. Idempotent replay keyed by receipt hash makes recovery boring.

SKIP THESE, FOR NOW

- ✗ The risk model**
Forty DAGs and eight months of history is not a training set. Revisit at a thousand runs a day.
- ✗ Canary DAG promotion**
With 40 DAGs and one repo, a git revert takes ninety seconds. Buy it later.
- ✗ Lineage-aware rollback**
You need point-in-time snapshots first. Until then, replay from raw and accept the minutes.

1 of 400

tenants affected, instead of the whole pipeline

3 min

to detect a payload change nobody announced

3 h

of receipts replayed, keyed by hash - no duplicates

2

engineers, no on-call rotation

Self-healing is not an enterprise feature. It is three patterns, and the small team needs them more — nobody is awake at 3 a.m.

Three failures we paid for, across both platforms

ATC · the retry storm

WHAT HAPPENED

Week 8. A breaker with a 2-minute cooldown, and 40 DAGs sharing one expired Infoflo credential, produced 11,000 task starts in an hour and knocked the scheduler over.

WHAT WE CHANGED

Cooldown is now exponential per DAG and jittered, and breaker actions themselves count against the hourly budget.

ATC · 2.1M duplicate rows

WHAT HAPPENED

Week 16. An automated replay hit a QA sink that used INSERT, not MERGE. The pipeline was idempotent on paper; the DDL had drifted a year earlier, and nobody re-checked.

WHAT WE CHANGED

Replay now refuses to run unless idempotency is proven by a recorded double-run test in CI - not by a config flag.

Recently · 11.11 quarantined itself

WHAT HAPPENED

Sale day. Receipt volumes moved 4 σ outside anything the scorer had seen, so it marked everything high risk and quarantined nine healthy merchant pipelines at peak.

WHAT WE CHANGED

Calendar-aware features for known sale events, a hard cap on quarantines per hour, and automatic shadow-mode fallback the moment input drift is detected.

A 90-day path that does not require permission for autonomy

DAYS 0-30

Instrument

- 1 Point a read replica at the metadata DB and build the feature query from slide 12
- 2 Add DAG fingerprints to your parse hook - 40 lines, zero risk
- 3 Write contracts for your ten most-depended-on tables
- 4 Log what the on-call actually did for every failure: that is your training label

Costs nothing. Changes no behaviour.

DAYS 31-60

Contain

- 1 Tier your pools and give quarantine its own lane
- 2 Ship circuit breakers — the highest value-per-line code in this talk
- 3 Turn on canary promotion for bundle changes
- 4 Run the risk model in shadow and score it against the humans

Reduces blast radius before any autonomy exists.

DAYS 61-90

Automate

- 1 Enable exactly two actions first: retry-with-reroute and defer-for-late-upstream
- 2 Add budgets and never_touch before the third action
- 3 Selective backfill once idempotency is proven in CI
- 4 Put a kill switch on the wall and test it in a game day

Autonomy, earned in small increments.

Should this heal itself?

- 1 A Spark task is OOM-killed at 02:00. The same task succeeded on a smaller input yesterday, and the sink is a MERGE.

AUTO-HEAL

- 2 The vendor adds a nullable promo_code column to orders. Nothing failed - the contract check flagged it.

AUTO-HEAL

- 3 Row counts in orders_fact drop 40%. A dbt model merged this morning; every task is green.

AUTO, WITH LIMITS

- 4 finance_gl shows duplicate journal entries after last night's automated replay.

WAKE A HUMAN

Three of the four heal themselves. The fourth is the one that decides whether anyone lets you keep the other three.

The rule behind each verdict

1	AUTO	OOM on a MERGE sink	Transient, idempotent, reversible. Retry with a larger pod and reroute the pool. The only nuance: count it, because three of these in a week is a sizing bug, not a transient.	slide 15
2	AUTO	additive schema drift	Rank 0 on the severity ladder. Evolve the target with add-columns-only and replay the partition. Nothing downstream reads a column that did not exist.	slide 11
3	AUTO, LIMITED	green runs, wrong numbers	This is a deploy problem, so the action is a canary rollback - pin the bundle, clear the affected runs. Restoring data is a separate decision and needs approval.	slide 16
4	HUMAN	not because it is finance	Because the sink proved non-idempotent - the replay itself caused this. never_touch would have blocked it anyway, but the real failure was automating against an unverified idempotency claim.	slide 27

Every one of these rules is four lines of YAML. The judgement is in knowing which scenario is which.

Takeaways

1

Detection is the cheap half of the win

Nine of our nine-hour outage was finding it. Contracts and fingerprints cost a week and cut MTTR in half before anything was automated.

2

Contain before you cure

Pools, quarantine lanes and circuit breakers stop one bad DAG from becoming a platform incident. Highest value per line of code you will write.

3

A bounded action vocabulary makes autonomy safe

Twelve typed, idempotent, reversible actions in reviewed policy - not a model improvising in production.

4

Verify once, then escalate

There is no second-order autonomy. A failed remediation is always a human's problem, with the full decision trace attached.

5

Build the control plane out of Airflow

It is already a scheduler with retries, RBAC, audit and a UI. Do not operate a second system to babysit the first.

Airflow stops being a scheduler the day it starts making decisions about itself.

Thank you

Questions - especially the uncomfortable ones about autonomy.

Kumuda Sreenivasa
Founder & Sr Data Architect
Receitly & ATC Drivetrain

Sandeep Bommisetti
Director
IAM ,NielsenIQ